

THE AI TOOLBOX · CLASS 11

Classification (Logistic Regression)

Predict **categories**, not quantities — the workhorse of applied ML.

Merit Point Academy · 60 min · press **S** notes · → next ·  to comment

RECAP · WHERE WE LEFT OFF

Last class in 60 seconds

A model is a learned relationship

Features in (X), a target out (y) — and "learning" is just fitting that relationship to data.

Loss + gradient descent

Loss measures wrongness; training walks downhill on it until it stops falling.

Judge on test data only

And never trust a prediction far outside the range you trained on.

Homework share: where did your model break when you pushed a feature past its training range? Today keeps *all* of that machinery and changes exactly one thing: **the target is now a category, not a number**. Distance in metres → real object, yes or no.

Where we're going

- **0–8'** Homework share + the machine the press said would walk and talk
- **8–16'** Categories vs. numbers · the sigmoid: turning a score into a probability
- **16–26'** The decision boundary — what the model actually learned
- **26–34'** Why accuracy lies, and what to report instead
- **34–42'** Demos: fit a classifier in Colab · the AI threshold advisor
- **42–54'** Two labs: logistic regression from scratch · the AV detector, with its threshold moved
- **54–60'** Your yes/no predictor + boundary plot, homework & wrap-up
- Quizzes &  comments throughout

By the end: explain binary classification, read a decision boundary, and know why a 96%-accurate model can be useless.

Quick check · tap an answer

Your camera sees a blob. Is it a real pedestrian, or a false alarm? You could feed the detector's confidence score into last class's linear regression. Why doesn't that work?

It works fine — just round the prediction to 0 or 1

A line's output runs from $-\infty$ to $+\infty$, so it can predict -3.2 or 7.9 — which are not probabilities and not categories

Linear regression can't handle more than one feature

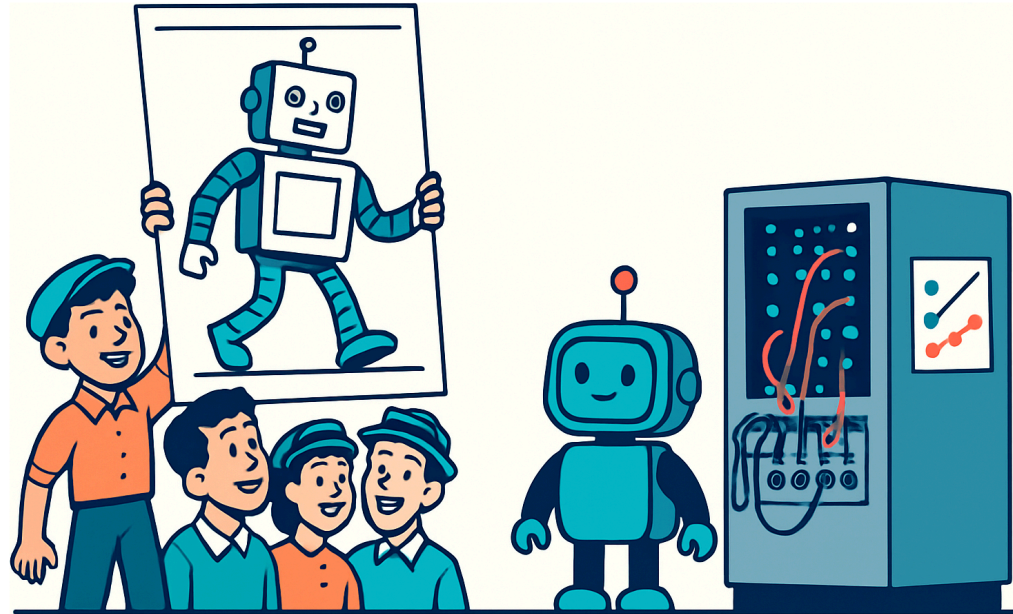
Because confidence scores are already probabilities, so no model is needed

The machine the press said would walk and talk

In **1958**, Frank Rosenblatt unveiled the **Perceptron**, and the New York Times reported it was the embryo of a computer expected to "walk, talk, see, write, reproduce itself and be conscious of its existence." It did none of those things. Underneath the headlines the Perceptron was a simple classifier that draws a **straight line between two groups of points** — very nearly what you'll build today. The hype collapsed, funding vanished, and the field entered its first winter. The humble idea survived it, and is now inside almost everything.

[Speak this story](#)

Neural voice · click to play / pause



CONCEPT 1 · THE SHAPE OF THE PROBLEM

Predicting a category



Regression (Class 10)

The target is a **number on a scale** — 27.4 metres, \$412,000, 68 points.
"How much?"



Classification (today)

The target is a **label from a set** — real object / false alarm, spam / not spam, benign / malignant. "Which one?"

KEY IDEA — logistic regression doesn't output a label directly. It outputs a **probability between 0 and 1**, and *you* choose the cut-off that turns it into a decision. That extra step looks like a technicality. It is the most consequential design choice in the whole model.

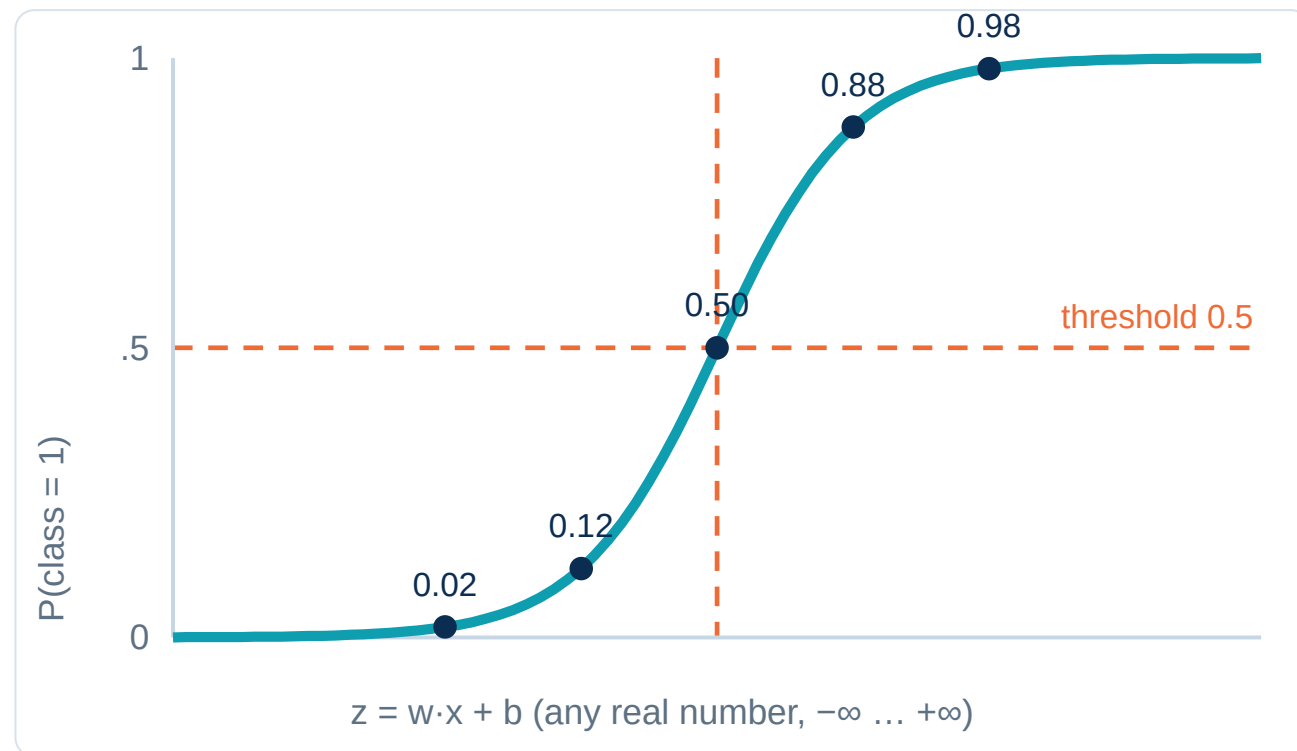
The sigmoid: a score becomes a probability

Keep Class 10's line. Feed its output — call it z — through one S-shaped squashing function, and whatever comes out is between 0 and 1.

$$z = w \cdot x + b$$

$$P(\text{class} = 1) = \sigma(z) = 1 / (1 + e^{-z})$$

$z = 0 \rightarrow$ exactly 0.50, maximum uncertainty. Large positive $z \rightarrow$ confidently 1. Large negative $z \rightarrow$ confidently 0. It never actually reaches 0 or 1, which is the model's way of never being certain.



One more thing changes with it: squared error stops being the right loss. Classification uses **cross-entropy** (log loss). It punishes confident wrong answers without limit — and, crucially, it still **pushes** when the model is confidently wrong. Squared error through a sigmoid does the opposite: at $P = 0.01$ on a true 1, its gradient is **50× smaller** than cross-entropy's, so training crawls exactly where it most needs to move.

Quick check · tap an answer

Your classifier returns $P(\text{real object}) = 0.52$ for a detection. What has the model told you?

It is 52% sure — a near coin-flip that happens to land on 'real object' at the usual 0.5 threshold

It is definitely a real object

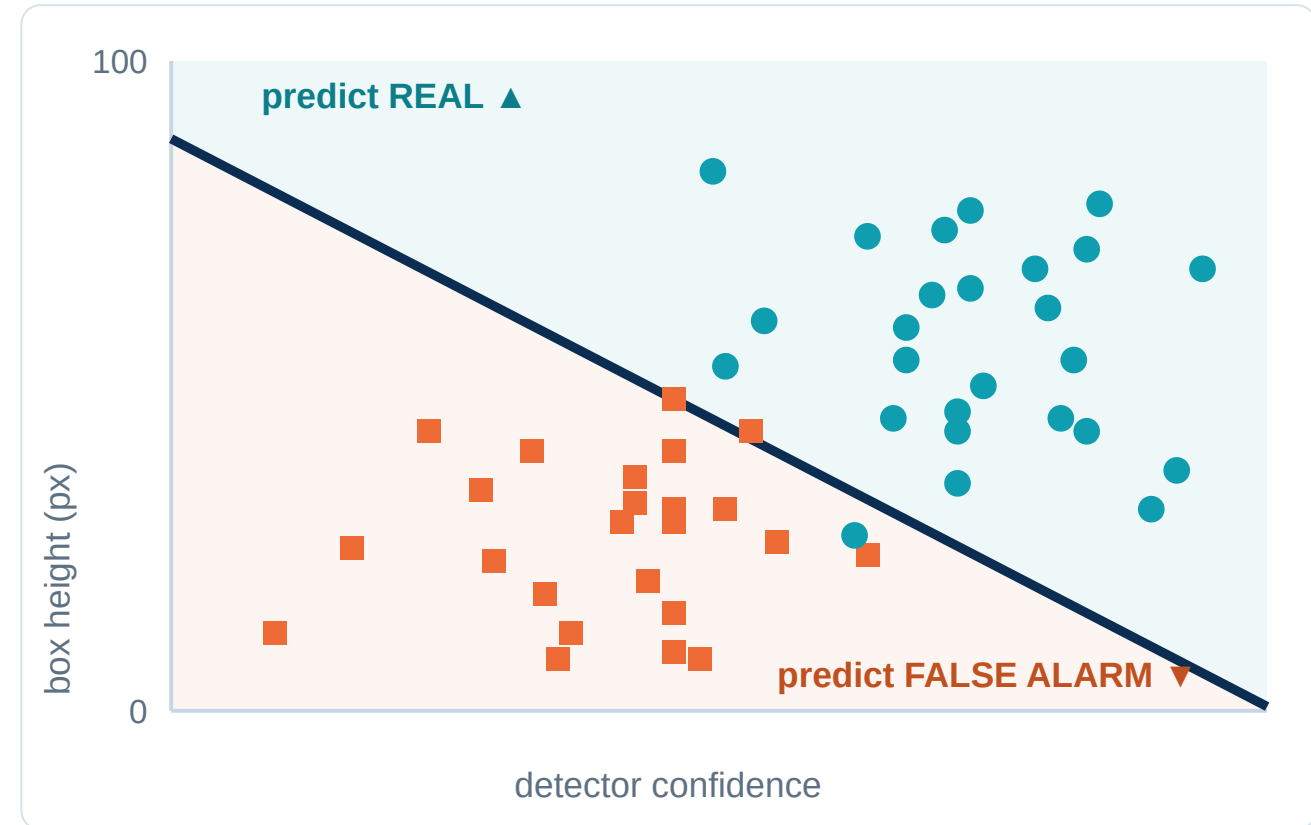
The model failed and produced garbage

52% of similar detections were real, which is a different claim entirely

The decision boundary

Set $\sigma(z) = 0.5$ and you're setting $z = 0$ — and $z = 0$ is just a straight line through your feature space. Everything on one side is predicted **real**; everything on the other, **false alarm**.

- **The boundary IS the model.** Two weights and an intercept, drawn.
- Points near the line are the 0.5-ish, uncertain ones
- Move your threshold off 0.5 and the line **slides** — same model, different caution



• real objects · ■ false alarms · the line is fitted at build time by gradient descent on all 50 rows of the detection log — it agrees with scikit-learn's LogisticRegression to three decimals: $2.24 \cdot \text{confidence} + 2.17 \cdot (\text{height}/100) - 2.25$, with 47/50 points on the correct side.

96% accurate and completely useless

200 dashcam frames. **8** contain a pedestrian. Ship a model that answers "no pedestrian" every single time and never looks at the image:

```
accuracy = 192 / 200 = 96.0%  
pedestrians found = 0 / 8
```

Nothing is broken. The metric is simply answering a question nobody asked.

Report these instead

Recall — of the real pedestrians, what fraction did we catch? (*here: 0.00*)

Precision — of the alarms we raised, what fraction were real?

Confusion matrix — all four boxes: hits, misses, false alarms, correct rejections.

Whenever one class is rare, accuracy flatters. Rare is the normal case in anything that matters — disease, fraud, equipment failure, pedestrians. Class 14 gives these metrics a full hour; today the rule is simply: **never report accuracy alone on imbalanced data.**

Real object, or false alarm?

Every frame the car processes, this question gets asked a few dozen times — and a wrong answer costs something different in each direction.

1

Features

Detector confidence, bounding-box height

2

Label

Real object (1) vs. false alarm (0)

3

Boundary

Plot it — then move the threshold and watch it slide

The asymmetry is the whole engineering problem. A false alarm means the car brakes for a plastic bag: annoying, occasionally dangerous for the driver behind. A miss means the car doesn't brake for a person. Those two errors are not worth the same, so **0.5 is a default, not a decision** — and no amount of extra training data will choose it for you.

DISCUSS · 5 MINUTES · WHICH MISTAKE IS WORSE?

A wrong yes, or a wrong no?

Name a real decision where a wrong **yes** costs far more than a wrong **no** — and one where it's the other way round.

Pick your case

Cancer screening? Spam filters? University admissions? Airport security?
Loan approvals? Say who pays for each kind of error.

Then the hard part

Should the model lean one way **on purpose**? And **who** should get to decide where the line goes — the engineer, the company, the regulator, or the person affected?

Pairs, 3 minutes — best example and best "who decides" answer in .

LIVE DEMO 1

Fit a classifier in Colab, five lines

The same shape as last week's regression demo — three of the five lines are literally identical. Then the two that aren't: probabilities, and the confusion matrix.

```
You ▶ X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,  
random_state=42, stratify=y)
```

```
You ▶ clf = LogisticRegression(); clf.fit(X_train, y_train)
```

```
You ▶ probs = clf.predict_proba(X_test)[: , 1] # ← the number the label came from
```

```
You ▶ print(confusion_matrix(y_test, clf.predict(X_test)))
```

```
You ▶ print(classification_report(y_test, clf.predict(X_test))) # precision, recall, per  
class
```



colab.research.google.com

LIVE DEMO 2

The AI threshold advisor

Class 4's mentor hat again, aimed at the one decision the model cannot make for you.

You ▶ I'm classifying pedestrian / no-pedestrian for a car. Missing a pedestrian is catastrophic; a false alarm is annoying. Should my threshold be above or below 0.5, and what do I lose?

You ▶ My model is 96% accurate but recall is 0.0. Explain to my teacher what went wrong.

You ▶ Explain precision and recall using my project's data, in two sentences a parent would understand.

You ▶ What could make my model's probabilities untrustworthy even when its accuracy is high?



chatgpt.com

Logistic regression from scratch — watch the S-curve appear

No scikit-learn. The same gradient-descent loop you wrote last week, with two edits: a sigmoid on the output, and cross-entropy instead of squared error. Then print the curve it learned in ASCII.

```
1 import math
2
3 # The AV detection log, one feature: the detector's own confidence score.
4 conf = [0.45,0.74,0.84,0.91,0.54,0.52,0.35,0.61,0.78,0.77,0.60,0.23,0.69,
5         0.72,0.54,0.51,0.76,0.85,0.54,0.54,0.54,0.62,0.83,0.51,0.69,0.86,
6         0.57,0.50,0.54,0.76,0.44,0.68,0.93,0.87,0.29,0.77,0.46,0.72,0.39,
7         0.86,0.76,0.58,0.95,0.82,0.58,0.71,0.56,0.43,0.75,0.40]
8 label = [0,1,1,1,0,0,0,1,1,1,0,0,0,1,0,0,1,1,0,0,
9          0,0,1,0,1,1,1,0,0,1,0,1,1,1,0,1,0,1,0,1,
10         1,1,1,1,0,1,0,0,1,0]      # 1 = real object, 0 = false alarm
11 n = len(conf)
12
```

[▶ Run](#)[✨ Explain](#)[🐛 Debug](#)[⚡ Improve](#)[🤖 Mira](#)

Python loads on first Run

▶ Run the code to see the output here.

Two features, a real boundary — then move the threshold

scikit-learn, an honest test split, a confusion matrix, and the experiment that matters: sweep the threshold and watch which mistakes you trade for which.

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.model_selection import train_test_split
3 from sklearn.metrics import accuracy_score, confusion_matrix
4
5 # One row = one detection: confidence, box height (px), was it real?
6 conf = [0.45,0.74,0.84,0.91,0.54,0.52,0.35,0.61,0.78,0.77,0.60,0.23,0.69,
7         0.72,0.54,0.51,0.76,0.85,0.54,0.54,0.54,0.62,0.83,0.51,0.69,0.86,
8         0.57,0.50,0.54,0.76,0.44,0.68,0.93,0.87,0.29,0.77,0.46,0.72,0.39,
9         0.86,0.76,0.58,0.95,0.82,0.58,0.71,0.56,0.43,0.75,0.40]
10 box_h = [8,64,45,31,40,20,43,60,50,77,43,12,24,54,29,32,43,54,31,48,
11         15,26,62,36,73,71,83,29,9,46,18,27,37,78,25,65,12,59,34,43,
12         35,53,68,68,31,45,8,40,74,23]
```

[▶ Run](#)[🌟 Explain](#)[🐛 Debug](#)[⚡ Improve](#)[🤖 Mira](#)

Python loads on first Run

▶ [Run the code to see the output here.](#)

YOUR PROJECT

Build a yes/no predictor

Train a binary classifier, **draw the boundary it learned**, and report a number that can't lie to you.

- Pick a yes/no dataset — your own, or the detection log from Lab 2
- Choose **two** features so the boundary is plottable in 2D
- Fit a LogisticRegression on the training split
- Plot the points coloured by class, plus the decision boundary
- Report the **confusion matrix**, not just accuracy

Deliverable

A binary classifier plus a decision-boundary plot.

Success looks like

The two classes are visibly separated, you can say what the boundary means in one sentence, and you can name which errors your model makes.

BUILD IT NOW · HANDS-ON

Fit it, draw the line, then move it

1. Pick two numeric features and a yes/no target — check how many of each class you have **before** you fit
2. `train_test_split(..., stratify=y)` — keep the class balance in both piles
3. `LogisticRegression().fit()` on train only
4. Print the **confusion matrix** and the classification report on the test set
5. Scatter the points coloured by class, then draw the boundary: solve $w_1x_1 + w_2x_2 + b = 0$ for x_2
6. One sentence: which error does your model make more often, and does that suit the problem?

Colab

scikit-learn

matplotlib

ChatGPT / Claude

Instructor's rule: count your classes before you trust your score. If 90% of your rows are one class, your accuracy started at 90% before the model learned anything at all.

What a finished classifier looks like



The boundary

$$z = 1.74 \cdot \text{conf} + 1.84 \cdot (\text{box_h}/100) - 1.78$$

Two weights and an intercept — readable as "more confidence and a bigger box both push toward REAL."



The honest score

Test accuracy 0.867 *with its confusion matrix beside it* — 7 hits, 6 correct rejections, 1 false alarm, 1 miss. Not accuracy alone.



The plot

Points coloured by class, the boundary drawn through them, labelled axes and a title — a stranger sees the split without you narrating it (Class 9's rule).

The bar: a boundary you can read in one sentence, a confusion matrix rather than a lone percentage, a labelled plot that proves the split — and one sentence on which of your two error types you'd rather make, and why.

Today's vocabulary

Classification — predicting which category a row belongs to. **Binary** = exactly two.

Decision boundary — the set of points where the model is exactly 50/50; a line in 2D, a plane in higher dimensions.

Cross-entropy (log loss) — classification's loss function: heavy punishment for confident wrong answers.

Sigmoid $\sigma(z)$ — squashes any real number into (0, 1) so it can be read as a probability.

Threshold — the probability cut-off that turns 0.63 into a decision. Defaults to 0.5; should be chosen, not inherited.

Precision / recall — of my alarms, how many were real? / of the real ones, how many did I catch?

RECOMMENDED HOMEWORK

Before next class

1. **Finish your yes/no predictor** — classifier + decision-boundary plot + confusion matrix.
Colab link in Google Classroom.
2. **Move your threshold** to 0.3 and to 0.7. Report what happens to your two error types, and which setting you'd ship.
3. **One paragraph:** for your problem, is a wrong "yes" or a wrong "no" worse — and who should decide?

★ Stretch (optional)

Kaggle's **Titanic** competition — the classic first binary classifier. Predict survival from age, sex and class; note how much of the work is the Class-8 cleaning, not the model.

[kaggle.com · Titanic](https://www.kaggle.com/c/titanic)

Task 2 is the one that matters. A model with a threshold you chose deliberately — and can defend — is a different object from a model with a threshold you never noticed. Questions → 💬 on any slide.

TODAY'S LINE TO REMEMBER

**"The machine's answer is a probability.
The decision is yours."**

— the threshold is not a hyperparameter, it's a value judgement

Logistic regression will tell you 0.63. Nothing in the mathematics tells you whether 0.63 should mean "brake."



Speak this

WRAP-UP

Three things to keep

- Classification predicts a **category** — via a probability, through a sigmoid
- The **decision boundary** is the model, drawn; the **threshold** slides it
- **Accuracy alone can lie** — report the confusion matrix, especially when one class is rare

Exit ticket: give an example where a wrong "yes" is worse than a wrong "no". Post it in .

Next · Class 12 — models whose reasoning you can actually read: decision trees and random forests.

BACKUP MATERIAL

Extra slides — if there's time

Deeper dives and extra practice. Skip in a tight hour; pull up on demand or for a longer session.

More than two classes

one-vs-rest and softmax

Precision & recall, properly

the four boxes, named

Bonus quiz

one more check

What if there are more than two classes?

One-vs-rest

Train one binary classifier per class — "pedestrian vs. everything", "car vs. everything", "cyclist vs. everything" — and take whichever is most confident. Simple, and it's what sklearn often does under the hood.

Softmax

One model, one score per class, all squashed together so they sum to 1.0 — the sigmoid's big sibling. This is exactly the layer at the end of nearly every modern neural network, including the ones that classify images and pick the next word.

The code barely changes — pass `y` with three labels instead of two and `LogisticRegression` handles it. What *does* change: your confusion matrix grows from 2×2 to 3×3 , and "accuracy" gets even less informative.

Precision and recall, properly

The confusion matrix

	said NO	said YES
truly NO	TN	FP
truly YES	FN	TP

FP = false alarm. FN = a miss. They are not the same mistake and should never share a number.

The two questions

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

"When I raise the alarm, how often am I right?"

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

"Of everything I should have caught, how much did I catch?"

Lowering the threshold buys recall and spends precision. There is no setting that maximises both.

Our lab, live: threshold **0.3** → recall **1.00** (nothing missed), precision 0.53 (half the alarms are junk). Threshold **0.7** → precision **1.00** (every alarm correct) and five real objects missed. Push to **0.9** and recall hits 0.00 — the model has become the always-say-no model from concept 4. Same weights, same data, different values.

Quick check · tap an answer

A hospital's screening model reports 99.2% accuracy. The disease affects 0.5% of the people screened. What should you ask first?

Nothing — 99.2% is excellent, ship it

How many of the actual cases did it catch? A model that says 'healthy' to everyone scores 99.5% here

Whether they used enough training data

Whether accuracy was computed on the training set